

Test Plan - Local LLM Jira Test-Case Generator

Project: Local LLM Jira Test-Case Generator

Source specification: Chapter_03_Local_LLM_Generator/Finetune_Prompt.md

Prepared: 2026-08-21

Status: Draft - human approval required

Credentials: Not included in this document; use local configuration only.

1. Scope & Objectives

Validate the two-screen Streamlit application that accepts a Jira issue key, retrieves the issue, loads a local test-case template, generates a draft with Ollama or Groq, and renders the result in the chat screen.

In scope

- Chat screen: message input, Send action, progress/error states, and generated output rendering.
- Settings screen: Jira URL, Jira email, Jira token, provider selection, Groq key, save/load behavior, and secret masking.
- Jira REST integration: issue-key parsing, authentication, response mapping, and error handling.
- Template loading from the local templates/ directory.
- LLM routing: Ollama as the default, explicit Groq selection, and Ollama-unavailable fallback to Groq.
- Configuration persistence and version-control safety.
- Functional, integration, security, usability, compatibility, reliability, and focused performance checks.

Out of scope

- Creating or editing Jira issues.
- Writing test cases or automation for unrelated products.
- Training or re-downloading an Ollama model.
- Production-scale availability, multi-user tenancy, or cloud deployment unless later specified.

2. Requirements Analysis: Gaps & Questions

Requirement area	Status	Finding / question
Core workflow	Present	The required Jira-to-generated-test-case flow is defined.
Screens and deliverables	Present	Two screens and the named modules are specified.
Jira acceptance criteria	Missing	No Jira issue key or ticket body was supplied, so ticket-specific scenarios cannot be derived yet.
Input validation	Ambiguous	Define accepted Jira-key format, whitespace behavior, and handling of multiple keys in one message.
Jira permissions	Ambiguous	Define required Jira scopes and expected behavior for private, missing, or inaccessible issues.
LLM output contract	Ambiguous	Define required fields, format, maximum response size, and behavior for malformed or incomplete output.
Ollama fallback	Ambiguous	Define timeout, retry count, and which failures trigger Groq fallback.
Performance	Missing	Define acceptable Jira fetch, first-response, and complete-generation response times.
Security	Ambiguous	Confirm encryption or OS-level protection for persisted secrets and whether logs may contain request/response content.

Persistence	Ambiguous	Define config-file location, permissions, corruption recovery, and behavior when settings are incomplete.
Compatibility	Missing	Confirm supported Python, Streamlit, Windows versions, browsers, and viewport sizes.
Rate limits / cost	Missing	Define Jira and Groq rate-limit handling, user messaging, and usage controls.
Accessibility / i18n	Missing	Confirm keyboard navigation, labels, contrast, screen-reader expectations, and supported locales.
Auditability	Missing	Confirm whether provider choice, request status, and generated drafts require local audit logs.

3. Test Scenarios

Priority meanings: P0 = release-blocking core path or credential exposure risk; P1 = important supported behavior; P2 = quality, resilience, or secondary coverage.

ID	Priority	Scenario	Expected result	Traceability
TP-001	P0	Launch the application with no saved settings.	Chat and Settings screens load; no secret is displayed; the user is directed to configure required values.	Prompt: two-screen app; settings flow
TP-002	P0	Save a valid Jira URL, email, token, Ollama provider, and model defaults.	Settings persist, token/key fields remain masked, and a reload restores non-secret configuration without exposing secrets.	Prompt: Settings; no hardcoded credentials
TP-003	P0	Enter a Jira issue request such as create test cases for QA-102.	The exact Jira key is parsed and the request enters the generation workflow.	Prompt: end-to-end flow 1
TP-004	P0	Generate test cases for a valid accessible Jira issue while Ollama is available.	Jira summary, description, and acceptance criteria are fetched; the local template is loaded; Ollama gemma3:1b is called; structured output appears in chat.	Prompt: end-to-end flow 2-5
TP-005	P0	Inspect source, repository status, and runtime logs after configuring credentials.	No Jira token or Groq key is hardcoded, rendered, or written to normal logs; local config is excluded from version control.	Prompt: credential guardrail
TP-006	P0	Make Ollama unavailable, then submit a valid Jira request with a Groq key configured.	The app reports the local-provider failure appropriately, calls Groq as fallback, and renders the generated result.	Prompt: mandatory fallback
TP-007	P1	Explicitly select Groq and submit a valid Jira request.	Groq is used directly; Ollama is not called.	Prompt: explicit provider opt-out
TP-008	P1	Submit an unknown, malformed, lowercase, or whitespace-padded issue key.	Input is validated consistently, and a clear actionable error appears without an LLM call.	Gap: input validation
TP-009	P1	Jira returns 401/403, 404, 429, 5xx, timeout, malformed JSON, or an empty description.	The app shows a safe, specific error, does not expose credentials, and does not generate from fabricated ticket data.	Prompt: fetch details; gaps
TP-010	P1	Jira issue has missing, empty, or unusually long acceptance criteria.	The app preserves available data, indicates missing content, applies defined size limits, and does not silently invent acceptance criteria.	Prompt: acceptance criteria; guardrails
TP-011	P1	Template directory is missing, empty, unreadable, or contains invalid template content.	A clear configuration error is shown; no partial or misleading test plan is presented.	Prompt: local template loading
TP-012	P1	Ollama returns timeout, connection refusal, rate limit, malformed output, or a partial response.	Defined retry/fallback behavior occurs once; failure is visible and no duplicate uncontrolled requests are made.	Prompt: fallback; gap

| TP-013 | P1 | Groq is selected or reached as fallback without a Groq key. | The app does not make an unauthenticated request and explains how to configure the missing key. | Prompt: Settings; security |

| TP-014 | P1 | Refresh the app, restart Streamlit, and reload after saving settings. | Intended settings persist; secrets remain masked; corrupted or missing config fails safely with recovery guidance. | Prompt: persist settings |

| TP-015 | P1 | Send repeated requests and press Send during an active request. | Controls prevent accidental duplicate submissions or clearly serialize them; chat history remains coherent. | Gap: concurrency / UX |

| TP-016 | P1 | Verify chat output for prompt injection text inside Jira description or acceptance criteria. | Ticket content is treated as untrusted data; system instructions and credential boundaries are preserved. | Security risk: external content |

| TP-017 | P1 | Use keyboard navigation and a screen reader on both screens. | All controls have accessible names, logical focus order, visible focus, and usable error messages. | Gap: accessibility |

| TP-018 | P1 | Use supported desktop browsers and narrow viewport sizes. | Layout remains usable; fields and generated output do not overlap or become inaccessible. | Gap: compatibility |

| TP-019 | P2 | Measure Jira fetch, Ollama/Groq first response, and complete generation time for small and large tickets. | Results meet agreed thresholds or show a progress/timeout message. | Gap: performance |

| TP-020 | P2 | Simulate provider recovery after a fallback and submit a later request. | Provider routing follows the configured policy and does not remain stuck on Groq unexpectedly. | Gap: reliability |

| TP-021 | P2 | Confirm generated output uses the local template structure and contains no unsupported claims. | Output is traceable to Jira content and clearly identifies assumptions or missing requirements. | Prompt: template merge; guardrails |

| TP-022 | P2 | Verify Jira/Groq rate-limit responses and repeated failures. | Backoff/user messaging follows agreed policy and avoids uncontrolled API cost. | Gap: rate limiting / cost |

4. Test Data & Environment

Required test data

- One accessible Jira issue with complete summary, description, and acceptance criteria.
- One issue with missing acceptance criteria.
- One private/inaccessible issue, one nonexistent issue, and one issue with long content.
- A valid and an invalid Jira key.
- A valid test-case template, plus missing and malformed template fixtures.
- Ollama available at <http://localhost:11434> with the existing gemma3:1b model; do not download or re-pull it.
- A dedicated non-production Groq key for explicit-provider and fallback tests.

Environment

- Windows developer workstation.
- Python and dependency versions to be pinned in requirements.txt.
- Streamlit application run locally.
- Supported browser matrix to be confirmed by the owner; begin with current Edge and Chrome.
- Jira credentials supplied through local environment/configuration only. Never place credentials in this plan, source files, screenshots, or test evidence.

5. Test Strategy

Execute in this order: configuration smoke tests, Jira integration tests, Ollama happy path, explicit

Groq path, fallback/error paths, security checks, accessibility/compatibility checks, then focused performance and exploratory testing.

Use equivalence partitioning for provider and input states, boundary-value analysis for ticket/output sizes and timeouts, decision tables for provider routing, and state-transition coverage for request progress, success, failure, and retry states.

Maintain traceability from each confirmed Jira acceptance criterion to one or more scenarios. Until the Jira ticket is supplied, TP-001 through TP-022 are specification-based and must be refined against actual ticket criteria.

6. Defect Reporting

Log defects in Jira with environment, provider, issue key (redacted where appropriate), reproduction steps, expected/actual results, severity/priority, screenshots, and sanitized logs. Never attach tokens, API keys, authorization headers, or unredacted sensitive ticket content.

7. Entry & Exit Criteria

Entry

- Application modules and template are available.
- Test environment and supported versions are documented.
- A non-production Jira issue and safe test credentials are available.
- Ollama model availability is confirmed.
- Ticket-specific acceptance criteria have been supplied and reviewed.

Exit

- All P0 scenarios pass.
- No open critical/high security defects or credential exposure.
- P1 failures are resolved or explicitly accepted by the test owner.
- Provider routing, persistence, error handling, and regression checks are evidenced.
- Results and known limitations are recorded.

8. Risks & Mitigations

| Risk | Mitigation |

|---|---|

| Exposed Jira token in the shared attachment | Revoke/rotate it immediately; remove it from chat history and any captured evidence where possible. |

| Jira ticket requirements are unknown | Obtain the issue key/body and update traceability before sign-off. |

| LLM output is nondeterministic | Validate structure and required fields; retain sanitized prompts/results for repeatability. |

| Hosted fallback leaks ticket data or incurs cost | Require explicit configuration, show provider state, minimize payloads, and test rate limits. |

| Local config contains secrets | Use restrictive file permissions, masking, .gitignore, and secret-scanning checks. |

| Dependency or browser differences | Pin dependencies and execute the confirmed compatibility matrix. |

9. Deliverables

- This draft test plan in Markdown.
- PDF export of this draft.
- Scenario execution evidence and sanitized defect reports after implementation.
- Updated requirement-to-scenario traceability after the Jira ticket is identified.

---- HUMAN REVIEW GATE ----

Assumptions made: The requested scope is the application described in Finetune_Prompt.md, not a specific Jira feature. The app is local/internal, Ollama is the default provider, and Groq is only explicit or fallback use.

Cannot confirm yet: Jira issue key and acceptance criteria; supported Python/Streamlit/browser versions; response-time targets; fallback timeout/retry policy; persisted-secret protection; accessibility, localization, rate-limit, and audit requirements.

Approval required: A tester or product owner must provide the Jira issue key or ticket body, answer the open questions, and approve or edit this draft before it is treated as an approved plan or used to create test cases/automation.

Security action: The Jira API token included in the conversation is a credential exposure. Revoke it in Atlassian and create a replacement before any integration testing.